Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту

Лабораторна робота

На тему: «СТВОРЕННЯ ТА ТЕСТУВАННЯ ГЕНЕРАТОРА ПСЕВДОВИПАДКОВИХ ЧИСЕЛ»
з дисципліни: «Захист інформації»

Виконав:
студент групи ШІ-22
Климишин Данило
Викладач:
Яковина В.С.

Львів 2026

Мета:

Ознайомитись з джерелами та застосуванням випадкових чисел, алгоритмами генерування псевдовипадкових чисел та навчитись створювати програмні генератори псевдовипадкових чисел для використання в системах захисту інформації.

Теоретичні відомості:

Сучасна інформатика широко використовує випадкові числа в різних програмах – від методу Монте-Карло до криптографії. Ряд алгоритмів захисту мережі, заснованих на засобах криптографії, передбачає використання випадкових чисел. Ці застосування висувають дві вимоги до послідовності випадкових чисел: випадковість і непередбачуваність.

Джерелами дійсно випадкових чисел потенційно можуть бути фізичні генератори шумів, такі як імпульсні детектори іонізуючого випромінювання, газорозрядні лампи, конденсатори з втратами струму тощо. Однак такі пристрої можуть знайти доволі обмежене застосування в додатках для захисту інформації. Туту існують проблеми як з випадковістю, так і з точністю отриманих таким методом чисел, не кажучи вже про проблеми підключення такого роду пристроїв до кожної системи в мережі.

Тому криптографічні додатки зазвичай використовують алгоритмічні методи генерування випадкових чисел. Відповідні алгоритми є детермінованими і тому породжують послідовності чисел, які не є статистично випадковими. Однак, якщо алгоритм є достатньо хорошим, породжувані ним послідовності чисел витримують багато тестів на випадковість. Такі числа часто називають псевдовипадковими.

Генератор псевдовипадкових чисел – алгоритм, що генерує послідовність чисел, елементи якої незалежні один від одного і підлягають заданому розподілу.

Найбільш популярним алгоритмом для генерування псевдовипадкових чисел є алгоритм, запропонований Лемером, який називається методом лінійного порівняння.

Пропонується три критерії, за якими можна оцінити якість будь-якого

генератора псевдовипадкових чисел.

1) Функція генерації повинна бути функцією повного періоду, тобто функція повинна породити усі числа від 0 до перед тим, як числа почнуть повторюватись.

2) Створена послідовність повинна вести себе як випадкова. Насправді ця послідовність не буде випадковою, оскільки генерується детермінованим алгоритмом, але існує багато статистичних тестів, які можна використовувати для того, щоб оцінити ступінь випадковості поведінки послідовності.

3) Функція генерації повинна ефективно реалізовуватись в рамках 32-бітної арифметики.

Тестування генераторів псевдовипадкових чисел. До послідовностей псевдовипадкових чисел висувають наступні вимоги:

• випадковість – послідовність повинна бути випадковою в деякому визначеному статистичному сенсі

- однорідність розподілу: розподіл чисел в послідовності має бути однорідним, тобто частота появи в послідовності бітів одиниць і нулів повинна бути приблизно однаковою
- незалежність: жодне зі значень послідовності не повинно виводитися з інших значень

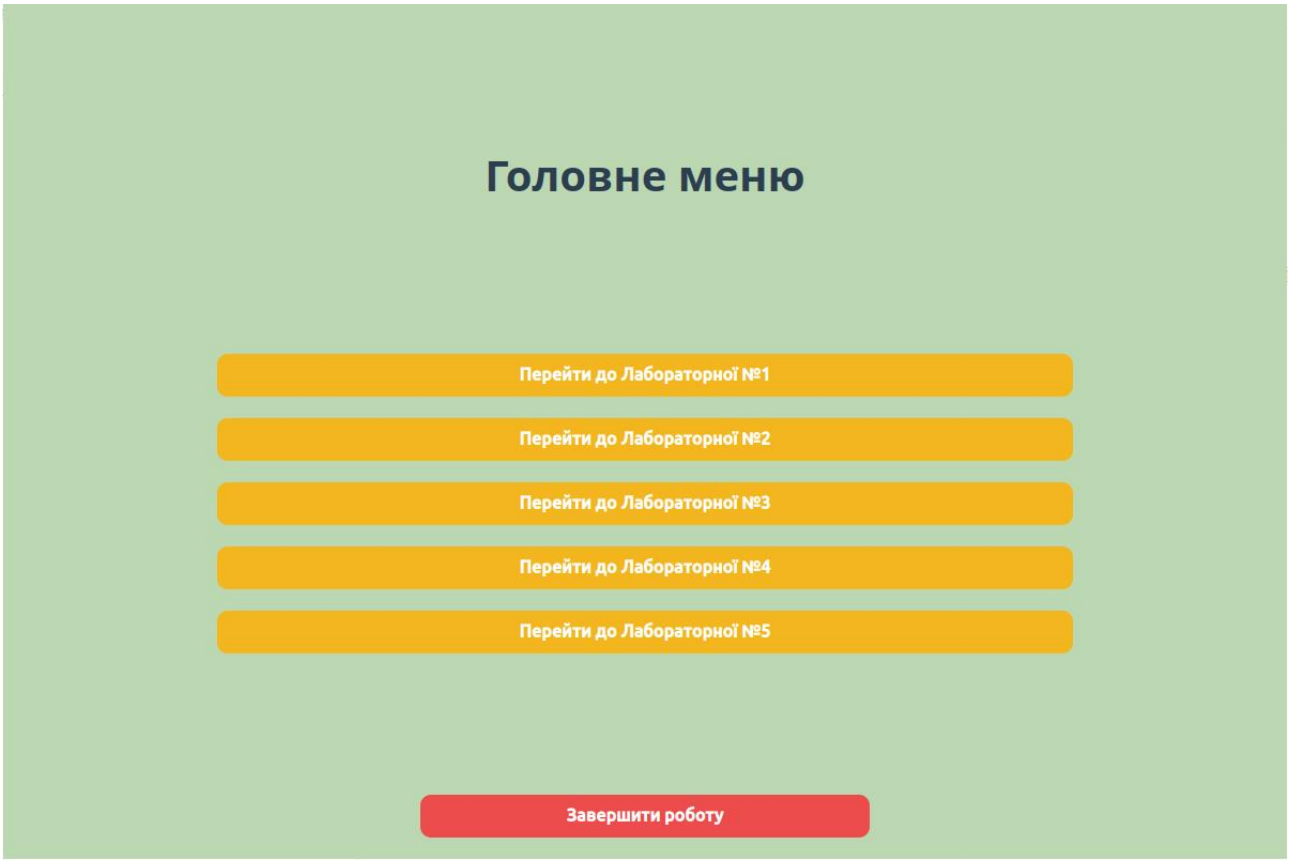
• непередбачуваність – чергові члени послідовності повинні бути непередбачуваними

Існують алгоритми для перевірки того, що деяка послідовність чисел відповідає заданому розподілу. Проте, алгоритм, що дозволяє "довести" незалежність, відсутній. Загальна стратегія полягає в застосуванні ряду тестів до досягнення певного рівня довіри щодо незалежності послідовності.

Хід роботи:

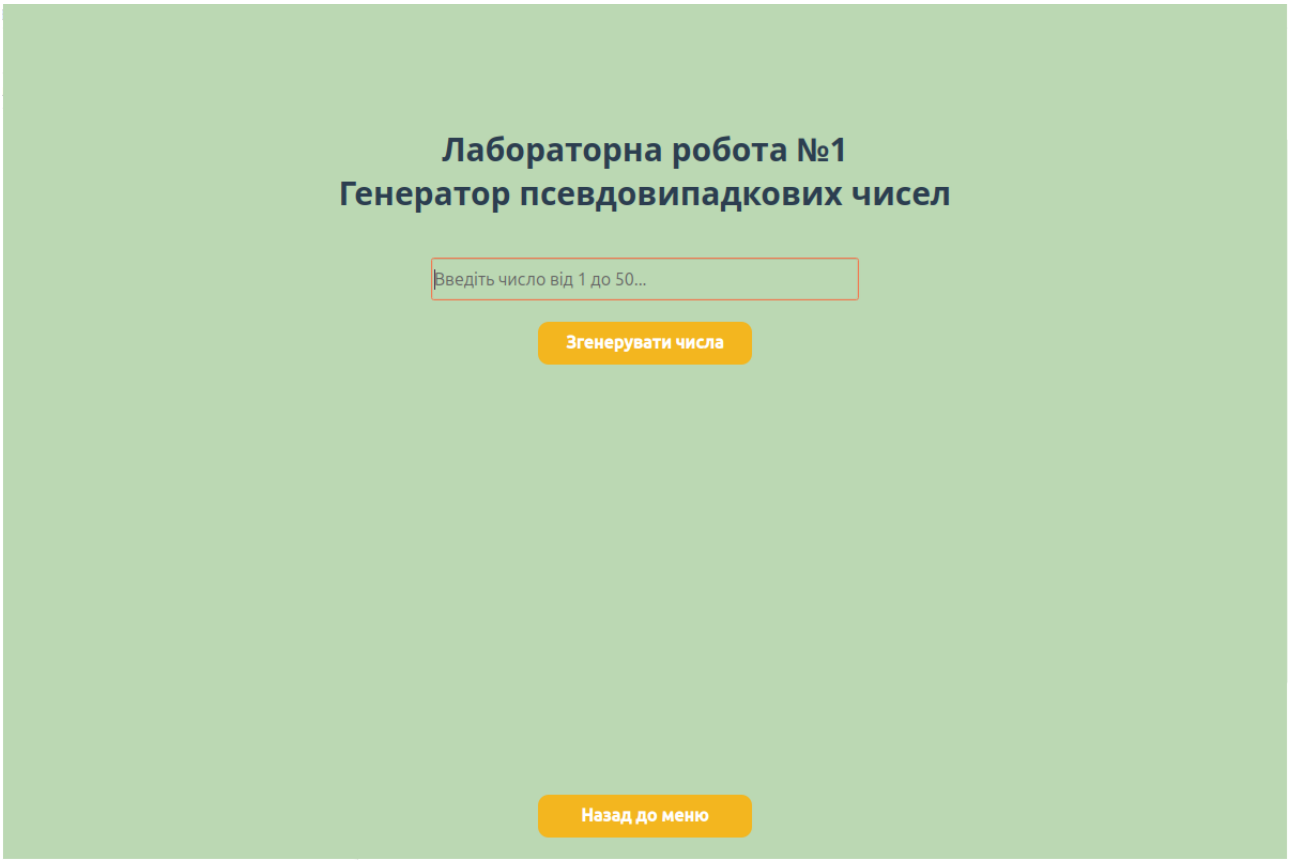
Варіант 3

Згідно до варіанту, наведеного в таблиці, створив програмну реалізацію генератора псевдовипадкових чисел за алгоритмом лінійного порівняння.



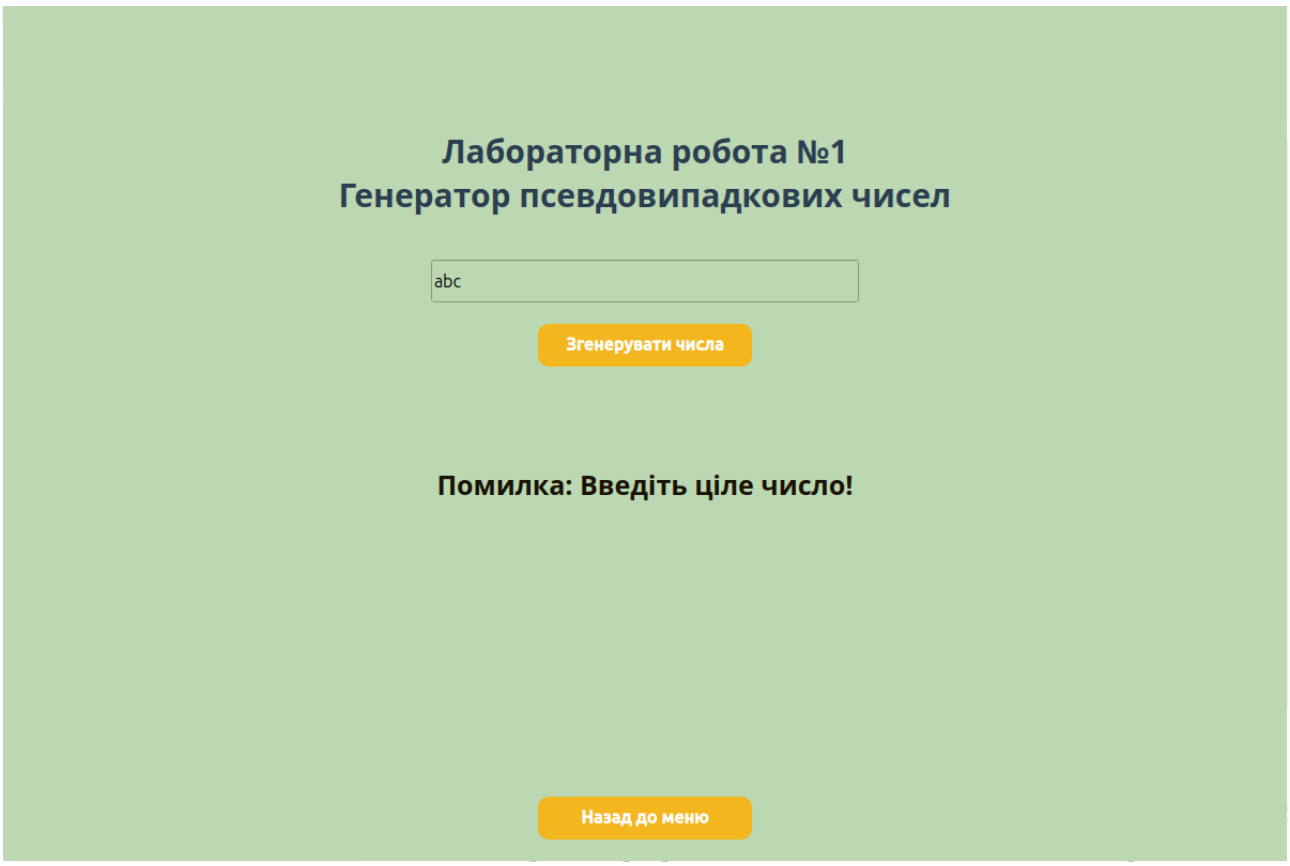
Головне меню програми

Я скористався бібліотекою PyQt6 для створення дизайну програми. Програма відкривається на сторінці головного меню де є активні кнопки переходу до лабораторних і кнопка завершити роботу програми.

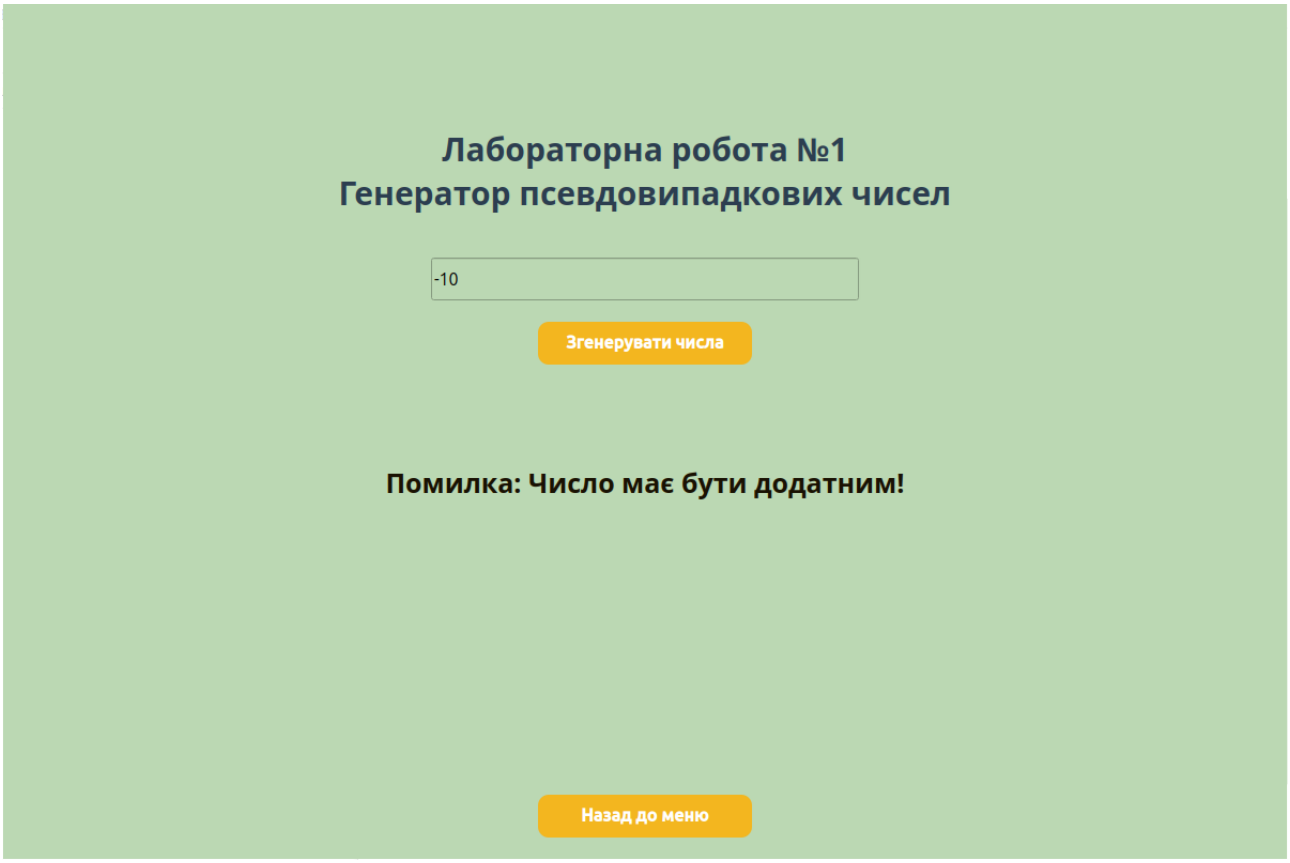


меню першої лабораторної роботи

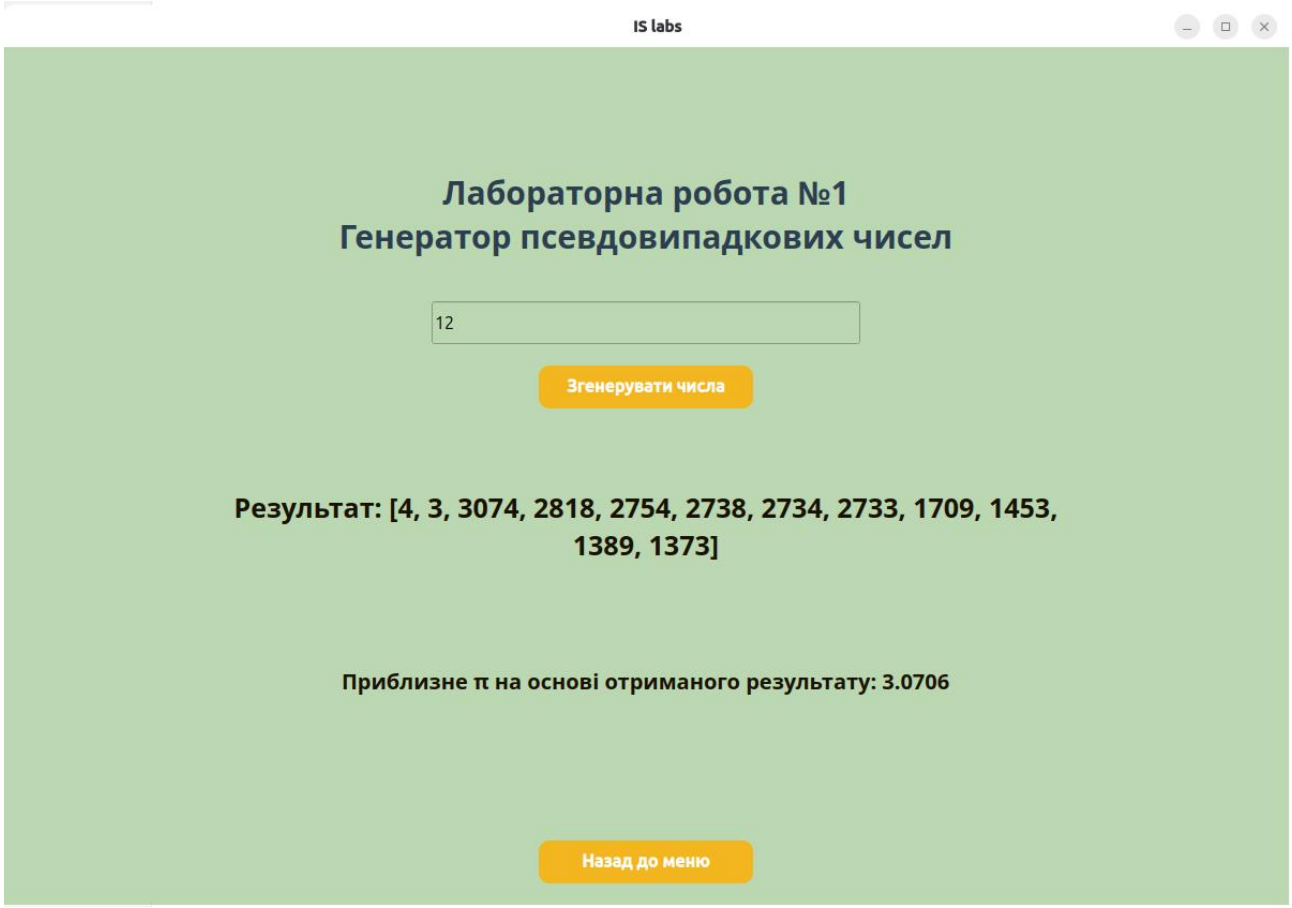
В меню першої лабораторної добавив текстове поле для введення числа, щоб передати його моїй функції генерування випадкових значеню. Програмою передбачено обробку некоректного користувацього вводу і обмежено діапазон можливих значень до 50 з метою запобігання переповнення поля для виводу чисел.



Результат спроби введення неправильного типу даних



Результат спроби введення від'ємного числа



результат роботи програми для числа 12

Для числа 12, програма згенерувала 12 псевдовипадкових чисел за лінійним алгоритмом. Нижче наведено приблизне рі отримане на основі послідовності випадкових чисел. На даному етапі, користувач має змогу вийте в головне меню або спробувати згенерувати числа за іншим вхідним значенням.

Open randomNumbers.txt ~/Documents/lolitex/3sem/IS/lab1/dist

```
----Results----
m = 4095, a = 1024, c = 2, x0 = 8
Number of generated integers: 12
X_1 = 4
X_2 = 3
X_3 = 3074
X_4 = 2818
X_5 = 2754
X_6 = 2738
X_7 = 2734
X_8 = 2733
X_9 = 1709
X_10 = 1453
X_11 = 1389
X_12 = 1373
Period: 18
```

Отримані значення також були записані в текстовий документ, включно з параметрами і програмно визначеним періодом послідовності

Як можна помітити період функції генерації всього 18 чисел, це є вкрай мало для використання генератора для задач з криптографії. Задля збільшення періоду і підвищення ефективності алгоритму, необхідно змінити початкові параметри.

```
def runTest(randomNumbers):
    counter = 0

def gcd(a, b):                          omNumbers) - 1):
    if not a > b:                       s[i], randomNumbers[i+1]) == 1:
        a, b = b, a
                                        domNumbers) - 1
    while True:
        r = a % b

        if r > 0:
            a = b                       rs
            b = r
            continue
        else:
            break
                                        (6 / r)
    return b
    return f'{estimated:..4f}'
```

Функція для тестування ГПВЧ на основі теореми Чезаро і допоміжна для неї функція пошуку найбільшого спільного кратного числа.

```python
class CompareTwoFunctionsOnEfficiency:
    def test_lemer_algorithm_w_random_library(self):
        numbersLemer = generateNumbers(20000)
        numbersRandom = [random.randint(1, 2**12-1) for _ in range(20000)]

        pi_val_lemer = runTest(numbersLemer)
        pi_val_random = runTest(numbersRandom)

        return pi_val_lemer, pi_val_random
```

функція для проведення тестування створеного алгоритму Лемера і функції генерування псевдовипадкових чисел із системної бібліотеки

Результат роботи функції для 20000 випадкових чисел:

```
Approximate pi value by Lemer algorithm: 3.4640
Approximate pi value by random algorithm: 3.1507
```

Як можна помітити, точність бібліотечної функції дуже висока, а точність мною створеного алгоритма менш точна за рахунок малого періоду послідовності.

Юніт-Тести:

```python
import unittest, random
from back_end import generateNumbers, runTest

class TestCryptoLogic(unittest.TestCase):

    def test_generate_numbers_count(self):
        count = 10
        result = generateNumbers(count)
        self.assertEqual(len(result), count)

    def test_generate_numbers_type(self):
        result = generateNumbers(5)
        self.assertIsInstance(result, list)
        for num in result:
            self.assertIsInstance(num, (int, float))

    def test_run_test_pi_range(self):
        numbers = generateNumbers(100)
        pi_val = runTest(numbers)
        self.assertTrue(2.0 <= float(pi_val) <= 4.0, f"Значення Pi ({pi_val}) занадто дивне")
```

Тут перевіряється кількість згенерованих чисел, їхній тип і приблизне значення числа pi.

```
----------------------------------------------------------------------
Ran 3 tests in 0.006s

OK
```

Результат перевірки юніт-тестів

```python
class randomNumbersGenerator:
    def __init__(self, m, a, c, x0):
        self.m = m
        self.a = a
        self.c = c
        self.x = x0

    def generateIntNumber(self):
        self.x = (self.a * self.x + self.c) % self.m
        return self.x

    def findPeriod(self):
        originalX = self.x
        seenValues = set()
        counter = 0
        while True:
            randomNumber = self.generateIntNumber()
            if not randomNumber in seenValues:
                counter += 1
                seenValues.add(randomNumber)
            else:
                break
        self.x = originalX
        return counter
```

реалізація лінійного алгоритму і функція пошуку періоду послідовності

Контрольні запитання:

1. **Вимоги до послідовності псевдовипадкових чисел**

Послідовність псевдовипадкових чисел (ПВЧ) генерується детермінованими алгоритмами, але повинна імітувати властивості істинно випадкової послідовності. Основні вимоги до неї включають:

- Детермінованість за початковим станом: За однакового початкового значення (seed) алгоритм повинен генерувати абсолютно ідентичну послідовність.
- Великий період: Послідовність не повинна повторюватися занадто швидко. Для сучасних задач довжина періоду має бути астрономічно великою.
- Ефективність обчислень:Алгоритм має генерувати числа швидко і з мінімальними витратами ресурсів процесора та пам'яті.

2. **Вимоги щодо випадковості послідовності псевдовипадкових чисел**

З точки зору статистики, послідовність має виглядати як білий шум. Основні вимоги:

- Рівномірність розподілу: Кожне число (або біт) із заданого діапазону повинно з'являтися з однаковою ймовірністю. Наприклад, у двійковій послідовності кількість нулів і одиниць має бути приблизно рівною.
- Незалежність (відсутність кореляції):Поява будь-якого числа в послідовності не повинна залежати від попередніх або наступних чисел. Знання частини послідовності не має давати статистичної переваги для визначення інших чисел.
- Відповідність постулатам Голомба:У двійкових послідовностях має зберігатися баланс одиниць і нулів, правильний розподіл серій (підрядкових однакових бітів) різної довжини, а автокореляційна функція має бути мінімальною.

3. **Пряма і зворотна непередбачуваність**

Ці концепції є критично важливими для криптографічно стійких генераторів псевдовипадкових чисел (CSPRNG).

- Пряма непередбачуваність (Forward Unpredictability): Зловмисник, знаючи всю згенеровану раніше послідовність і поточний стан системи, не може передбачити наступне число або біт з імовірністю, яка значно перевищує 50% (для двійкового генератора).
- Зворотна непередбачуваність (Backward Unpredictability / Backtracking Resistance): Якщо зловмисник якимось чином дізнався поточний внутрішній стан генератора, він не повинен мати змоги обчислити попередні стани та відновити числа, які були згенеровані в минулому.

4. **Властивості, які досліджують тести NIST SP 800-22**

Набір тестів NIST SP 800-22 є світовим стандартом для перевірки статистичної випадковості. Він включає 15 тестів, які перевіряють різні аномалії та шаблони:

- Частотні властивості (Frequency Test): Перевіряє співвідношення нулів і одиниць на відповідність рівномірному розподілу.
- Аналіз серій (Runs Test): Досліджує загальну кількість серій (послідовностей однакових бітів) для перевірки швидкості зміни між нулями та одиницями.
- Виявлення періодичності (Spectral DFT Test): Використовує дискретне перетворення Фур'є для виявлення періодичних властивостей (піків) у послідовності.
- Пошук шаблонів (Template Matching Tests):Перевіряє частоту появи наперед заданих комбінацій бітів (як з перекриттям, так і без).
- Стисливість (Maurer's Universal Statistical Test): Перевіряє, чи можна стиснути послідовність без втрати інформації. Істинно випадкова послідовність не піддається стисненню.
- Лінійна складність (Linear Complexity Test):Визначає довжину найкоротшого регістра зсуву з лінійним зворотним зв'язком (LFSR), здатного згенерувати цю послідовність. Якщо складність низька — послідовність вразлива.

5. **Що таке генератор псевдовипадкових чисел?**

Генератор псевдовипадкових чисел (ГПВЧ або PRNG) — це детермінований математичний алгоритм, який приймає на вхід початкове значення (так зване "зерно" або *seed*) і генерує нескінченну або дуже довгу послідовність чисел. Ці числа не є випадковими в строгому фізичному сенсі, оскільки їхнє обчислення повністю підпорядковане формулі, але їхні статистичні властивості імітують істинно випадкові числа.

6. **Що таке період послідовності псевдовипадкових чисел?**
Оскільки комп'ютер оперує скінченним об'ємом пам'яті, внутрішній стан будь-якого ГПВЧ має обмежену кількість можливих комбінацій. Відповідно, через певну кількість кроків алгоритм неминуче повернеться до стану, в якому він уже був. **Період послідовності** — це мінімальна кількість згенерованих чисел, після якої послідовність починає повністю повторювати саму себе. Сучасні генератори (наприклад, Mersenne Twister) мають періоди порядку $2^{19937} — 1$.

7. **Критерії оцінки якості генератора псевдовипадкових чисел**

- Довжина періоду: Чим більша, тим краще.
- Статистична бездоганність: Успішне проходження суворих статистичних тестів (NIST, Diehard).
- Продуктивність: Кількість тактів процесора на генерацію одного байта/слова.
- Розподіл у багатовимірному просторі: Якщо брати числа попарно або по три як координати точок, вони мають рівномірно заповнювати простір, а не шикуватися в площини.
- Стійкість до передбачення: (Лише для криптографічних) Неможливість відновити *seed* або поточний стан за згенерованим виходом.

8. **Параметри алгоритму лінійного порівняння для повного періоду**

Алгоритм лінійного порівняння (найчастіше відомий як Лінійний конгруентний генератор, ЛКГ) описується формулою: X_n+1 = (a * X_n + c) \ mod (m) де X — послідовність псевдовипадкових чисел, m — модуль, a — множник, c — приріст.

- Модуль m має бути простим числом. Найчастіше для цього використовують прості числа Мерсенна (наприклад, $2^{31}-1$, що ідеально підходить для 32-бітних систем).
- Множник a має бути первісним коренем за модулем m (primitive root modulo m). Це означає, що ступені числа a від 1 до m−1 при діленні за модулем m повинні генерувати всі можливі залишки від 1 до m−1 без повторень

9. **Переваги алгоритму лінійного порівняння (ЛКГ)**

- Екстремальна швидкість: Алгоритм потребує лише однієї операції множення, однієї операції додавання та однієї операції взяття модулю.
- Простота реалізації: Пишеться кількома рядками коду будь-якою мовою програмування.
- Мінімальне використання пам'яті: Для генерації наступного числа потрібно зберігати лише попереднє згенероване число (стан — всього одна змінна).

10. **Недоліки алгоритму лінійного порівняння**

- ❿ Повна криптографічна нестійкість: Достатньо знати лише кілька підряд згенерованих чисел (іноді достатньо трьох), щоб розв'язати систему рівнянь, визначити параметри a, c, m і передбачити всю подальшу послідовність.
- ❿ "Ефект Марсальї" (Marsaglia effect): При використанні чисел ЛКГ як координат точок у N-вимірному просторі виявляється, що всі точки лежать на порівняно невеликій кількості паралельних гіперплощин. Тобто просторовий розподіл є вкрай неякісним.
- ❿ Вразливість молодших бітів: Якщо модуль m є ступенем двійки (що часто роблять для оптимізації), молодші біти згенерованих чисел мають дуже короткі періоди (наприклад, наймолодший біт часто просто чергується 0-1-0-1).

Висновки:

У ході виконання першої лабораторної роботи було успішно спроєктовано та реалізовано програмний додаток для генерації псевдовипадкових чисел (ГПВЧ).

Під час розробки було дотримано ключових архітектурних вимог, зокрема реалізовано чітке розділення програми на користувацький інтерфейс (front-end) за допомогою фреймворку PyQt6 та обчислювальну логіку (back-end). Такий підхід ізолював алгоритмічну частину від візуальної та заклав надійну основу для подальшого масштабування проєкту й інтеграції наступних лабораторних робіт (MD5, RC5) у єдиний застосунок.

Особливу увагу було приділено UI/UX складовій: розроблено інтуїтивно зрозумілий інтерфейс із єдиним головним меню, зручною навігацією та механізмами обов'язкової валідації вхідних даних. Програма ефективно обробляє виняткові ситуації (введення некоректних символів, від'ємних чисел або перевищення лімітів), виводячи користувачу інформативні повідомлення про помилки.

Для оцінки якості роботи генератора було реалізовано статистичний тест (обчислення наближеного значення числа Пі на основі згенерованих послідовностей). Крім того, надійність та незалежність бекенд-модуля підтверджено написанням автоматизованих юніт-тестів (unit tests).

На завершальному етапі готовий програмний продукт було скомпільовано у самостійний виконуваний файл. Це забезпечило можливість його запуску без прив'язки до середовища розробки (IDE) чи наявності встановленого інтерпретатора Python.